HierCIM: A 16-Kb SRAM-Based Digital CIM Macro With Hierarchical Adder-Tree Accumulation for Edge CNN Inference

Vikash Vishwakarma[1], Gopal Raut[2], Amit Mittal[1], Naman Goyal[1], Baker Mohammad[2], Santosh Kumar Vishvakarma[1]

[1]Department of Electrical Engineering, Indian Institute of Technology Indore, India
[2]Department of Computer and Information Engineering, Khalifa University, Abu Dhabi, UAE

***Abstract*—The growing demand for efficient deep-learning inference on edge platforms requires hardware that is both energy-efficient and practically implementable. This work presents a 16-Kb all-digital SRAM-based compute-in-memory (CIM) macro for low-bit CNN inference, featuring a hierarchical adder-tree–based accumulation architecture. The design integrates a NOR-enabled SRAM compute cell, column-wise rearrangement network, sparsity-aware compression, and multi-stage hierarchical accumulation within a 64-bank 64×4 architecture, enabling scalable bit-serial processing and utilization-aware mapping. Implemented in 65-nm CMOS, the macro achieves 8.19 TOPS effective throughput at 1.0 V and a peak energy efficiency of 586 TOPS/W at 0.9 V under practical operating conditions. Hardware-compatible CNN mapping is demonstrated using LeNet-5, VGG-8, and ResNet-8. The design achieves 98.1% and 72.3% accuracy on MNIST and CIFAR-10, respectively, with 1-bit activations and 4-bit weights, while 4-bit configurations on deeper networks show only 3–4% degradation from FP32 baselines. CNN inference is evaluated using a hardware-compatible post-training quantized (PTQ) flow without retraining. These results demonstrate that the proposed SRAM-CIM architecture provides an efficient and scalable accumulation solution with a practical trade-off among throughput, energy efficiency, and implementability for edge-oriented DNN inference.**

***Index Terms*—Digital compute-in-memory, SRAM, low-bit CNN inference, banked architecture, energy-efficient MAC, edge AI accelerator.**

I. Introduction

Deep neural networks (DNNs) have enabled major advances in vision, speech, and intelligent sensing, but their deployment on edge platforms remains constrained by the high cost of data movement between memory and compute units. In conventional von Neumann systems, repeated transfers of activations and weights dominate both latency and energy consumption, especially for convolution-heavy workloads. Compute-in-memory (CIM) architectures [1], [2] address this bottleneck by embedding computation inside or near the memory array, thereby reducing off-array data movement and improving energy efficiency. Early CIM research emphasized analog implementations, where multiply–accumulate operations are performed through current summation or charge-domain accumulation. Although analog CIM [3], [4] offers high parallelism, its practical deployment is often limited by process variation, noise sensitivity, non-linearity, and the overhead of analog-to-digital conversion. Digital CIM [5], [6] avoids many of these limitations by operating on exact bit-level [7]–[10] representations, preserving compatibility with conventional CMOS design flows and enabling more predictable timing, verification, scaling, and real world application [11].

Owing to its reliability, energy efficiency, and architectural flexibility, DCIM has emerged as a leading paradigm for accelerating deep learning and other data-intensive workloads on edge AI platforms [12], [13]. Within digital CIM, SRAM-based approaches are especially attractive because SRAM is already widely used in system-on-chip platforms and supports fast read/write operation with mature design infrastructure. However, practical SRAM-based digital CIM still faces several challenges. First, a compute-enabled cell must be integrated without excessively disrupting array regularity. Second, the large number of bitwise outputs produced in parallel must be rearranged and reduced efficiently before accumulation. Third, the accumulation path must be kept short enough to maintain high throughput while avoiding excessive switching activity and peripheral overhead. Finally, the architecture must support a realistic mapping of CNN workloads rather than only isolated arithmetic demonstrations.

The proposed architecture, a 16-Kb SRAM-Based Digital CIM Macro With Hierarchical Adder-Tree Accumulation for Edge CNN inference, integrates a rearrangement network, sparsity-aware compression, and a hierarchical accumulation path within a banked SRAM-CIM macro to improve energy-efficient CNN inference. This work addresses these challenges through a banked all-digital SRAM-CIM macro targeted to low-bit inference. The design intentionally focuses on variable 1–4 bit input feature/activation and 4-bit weights, a regime that preserves digital robustness while keeping peripheral arithmetic cost manageable. The main contribution of the work lies in the integration and co-optimization of the compute-enabled SRAM array, rearrangement logic, sparsity-aware compression, compact arithmetic, hierarchical accumulation, and CNN mapping strategy. The proposed architecture intentionally targets the low-precision regime with 1–4 bit input feature and up to 4-bit weights. The goal is not to claim algorithm-level state-of-the-art accuracy on large-scale benchmarks, but to demonstrate a physically implementable SRAM-CIM macro and reduction datapath for compact, energy-efficient edge

Corresponding author: Santosh Kumar Vishvakarma (email: skvishvakarma@iiti.ac.in).

inference. The main contributions of this work are summarized as follows:

- A 16-Kb, 64-bank SRAM-based digital CIM macro that combines in-array bitwise multiplication, rearrangement, compression, hierarchical accumulation, and shift-and-accumulate within a unified all-digital dataflow.
- A low-switching accumulation path based on a rearrangement network and sparsity-aware 4:3 compressor, reducing redundant transitions before the adder tree.
- A compact arithmetic backend using a novel 38-transistor PTL-based 4-bit adder and a hierarchical accumulation scheme to improve throughput in low-bit MAC execution.
- A layout-aware implementation and a hardware-compatible post-training quantization (PTQ) inference flow are demonstrated, including layer-wise mapping for CNN models with configurable 1–4-bit activations and 4-bit weights.

The remainder of this paper is organized as follows. Section II discusses the background and motivation for low-bit SRAM-based digital CIM. Section III presents the proposed banked SRAM-CIM architecture and its physical implementation. Section IV describes the CIM architecture mapping, and quantization flow, Hardware-Compatible inference and presents the system-level performance results along with comparisons to prior work. Finally, Section V concludes the paper.

II. Background and Motivation

Digital compute-in-memory (DCIM) has emerged as an effective way to reduce the memory-access overhead of matrix-vector multiplications [14] in deep neural networks. In DCIM systems, multiplication is typically mapped to simple bitwise logic, while accumulation is performed in the memory periphery using digital arithmetic circuits. This all-digital operation offers deterministic precision, strong compatibility with standard CMOS processes, and improved resilience to process, voltage, and temperature variations compared with mixed-signal alternatives.

Among DCIM options, SRAM-based implementations are particularly appealing because they provide mature read/write infrastructure, high speed, and straightforward integration into digital systems. At the same time, the main difficulty in SRAM-CIM is not merely performing one bitwise operation inside the cell, but efficiently converting a large set of simultaneous bit-level outputs into a final MAC result [15]–[17]. As array size grows, this requires careful coordination among local compute logic, partial-product organization, compression, adder-tree design [18], [19], and control of bit-serial accumulation [20]. To further enhance efficiency, compressor circuits such as 3:2 and 4:2 compressors are integrated ahead of the adder tree, reducing the number of intermediate operands, lowering logic depth, and minimizing switching activity [21], [22]. The combined use of compressors and adder trees forms the computational backbone of scalable and energy-efficient DCIM designs [23], [24].

Recent work has demonstrated the benefits of compressor-assisted reduction and hierarchical adder trees for shortening the critical path of accumulation. Likewise, sparsity-aware gating has been shown to reduce switching activity in workloads with a high fraction of zero-valued operands. However, combining these ideas inside a banked SRAM-CIM macro while maintaining a regular physical implementation remains non-trivial. Added logic inside the memory cell increases area, peripheral arithmetic consumes routing resources, and wider precision directly increases accumulation cost. Recent advances extend these concepts by supporting higher-bitwidth and reconfigurable precision through hybrid bit-serial multiplication and parallel accumulation, enabling scalability without excessive latency overhead [25], [26]. Despite significant progress, challenges remain in efficiently handling sparsity, reducing arithmetic overhead, and minimizing dynamic power consumption in large CIM arrays [27]–[30]. Prior SRAM-based CIM efforts highlight persistent trade-offs among area, latency, and scalability [31], [32]. These limitations motivate the proposed all-digital DCIM approach, which emphasizes compact design, robustness, and scalability, establishing a strong foundation for next-generation AI and DSP accelerators.

The design proposed in this work is motivated by the observation that many edge-AI workloads can tolerate low-bit quantization when the hardware and mapping flow are co-designed appropriately. For this reason, the present work deliberately targets the low-precision inference regime, using 1-bit activations and up to 4-bit weights. In this operating region, digital SRAM-CIM can preserve robust logic-level behavior while achieving strong energy efficiency and scalable bank-level parallelism. The goal of the proposed architecture is therefore not to maximize arithmetic precision, but to provide a physically implementable and energy-efficient SRAM-CIM macro for practical low-bit CNN inference.

III. Proposed SRAM-Based DCIM Architecture

A. Macro Architecture Overview

The proposed SRAM-based DCIM macro consists of 64 identical CIM banks, each implemented as a 64×4 compute-enabled SRAM array, providing two levels of parallelism. First, 64 bitwise products are generated within each bank. Second, multiple banks operate concurrently to process different channels, kernels, or neuron groups, as shown in Fig. 1. Each bank produces a local partial sum. When banks are mapped to different output channels, their outputs remain independent. For partitioned computations, bank-level partial sums are rearranged to minimize switching activity and accumulated through a controller-managed inter-bank adder to generate the final output. Input activations are broadcast to all banks, while weights are stored locally within SRAM cells, enabling in-memory MAC operations and efficient dot-product computation with reduced data movement. The banked organization supports parallel processing across multiple channels, improving throughput and energy efficiency, while peripheral logic performs final accumulation and output generation.

The proposed SRAM-CIM organization introduces a trade-off between storage density and compute efficiency, characteristic of compute-in-memory architectures. The orthogonal input direction and compact row structure enable efficient bitline-based computation and facilitate integration of logic within the memory array. The design targets inference workloads with quasi-static weights, where weights are preloaded

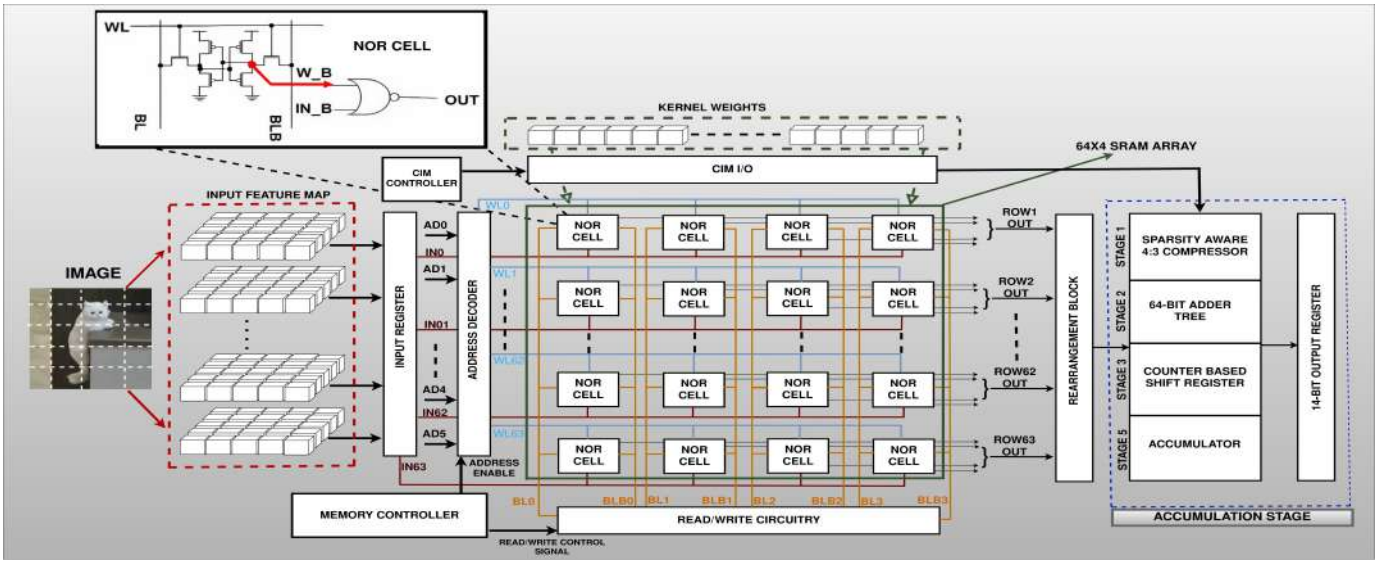


Fig. 1: Proposed 16-Kb SRAM-based digital CIM macro architecture. The macro comprises multiple banks with compute-enabled bit-cells that perform in-memory MAC operations, where input activations are broadcast and weights are stored locally. Peripheral circuits, including rearrangement/compression logic, a hierarchical adder tree, and SAC/control circuitry, support efficient accumulation and output generation.

and reused across multiple MAC operations, allowing weight-loading overhead to be amortized over many cycles. While the reduced row size and non-standard access direction are not optimal for conventional SRAM read/write operations, they enable higher compute parallelism and improved energy efficiency. Scalability is achieved through multi-macro deployment in parallel or tiled configurations. Alternative organizations, such as aligning the input direction with the bitline direction, may improve memory access efficiency but typically reduce compute parallelism or increase routing and control overhead. Therefore, the proposed organization represents a balanced design choice for edge-oriented DNN inference.

The proposed macro is optimized for low-bit arithmetic, with binarized activations and 4-bit weights in the reported inference experiments. This enables local bitwise multiplication within each bank, while only reduced partial products are forwarded to peripheral accumulation logic, significantly reducing switching activity and arithmetic complexity compared to higher-precision implementations while preserving digital robustness. At the top level, each bank integrates the compute-enabled SRAM array, input registers, read/write circuitry, rearrangement network, sparsity-aware compression, hierarchical adder tree, and shift-and-accumulate logic. A centralized controller manages bank activation, data movement, and operation sequencing. The banked architecture also supports selective activation, allowing inactive banks to be gated off under reduced workloads for improved energy efficiency.

B. *NOR-Enabled 6T SRAM Compute Cell and Peripheral*

The proposed CIM design is based on a modified 6T+4T SRAM cell, where four additional transistors are integrated into the conventional 6T structure to enable in-memory logic operations. These transistors form controlled pull-up and pull-down paths to realize NOR functionality, which is further leveraged to implement AND operations through input and weight complementation. The transition between memory and compute modes is implicitly governed by bitline/wordline biasing and input conditions, eliminating the need for explicit control signals (e.g., CIM ENABLE) and avoiding additional routing overhead. During standard memory operations, only the 6T storage nodes are active, while the compute transistors remain inactive. Post-layout PVT analysis indicates that the worst-case stability occurs at the SS corner under low supply voltage (0.9 V) and high temperature (125°C), where reduced carrier mobility weakens both pull-up and pull-down networks, degrading hold and read SNM. In contrast, the FF corner at high supply voltage (1.1 V) and low temperature (-40°C) provides the best-case stability, benefiting from enhanced carrier mobility and stronger drive strength. The 64×4 bank organization is deliberately optimized for compute throughput at the expense of conventional SRAM density, targeting inference workloads with quasi-static weights. In such scenarios, weights are loaded infrequently and reused across numerous MAC operations, thereby amortizing the weight-loading overhead.

TABLE I: Post-layout evaluation of SRAM stability metrics across process corners and supply voltages

V (V)	Hold SNM			Read SNM			Write Margin		
	TT	SS	FF	TT	SS	FF	TT	SS	FF
0.9	0.38	0.34	0.40	0.28	0.24	0.30	0.18	0.16	0.20
1.0	0.42	0.38	0.44	0.31	0.28	0.33	0.21	0.19	0.23
1.1	0.46	0.42	0.48	0.34	0.31	0.36	0.24	0.22	0.26

C. *Rearrangement Network and Sparsity-Aware Compression*

When a bank is activated, 64 bitwise products are generated in parallel from the compute-enabled cells. Directly feeding these into a wide accumulation tree would increase switching activity and routing overhead. To address this, a rearrangement stage groups the generated bits according to their significance prior to compression. As illustrated in Fig. 3, the partial

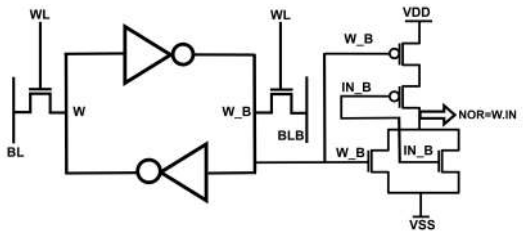
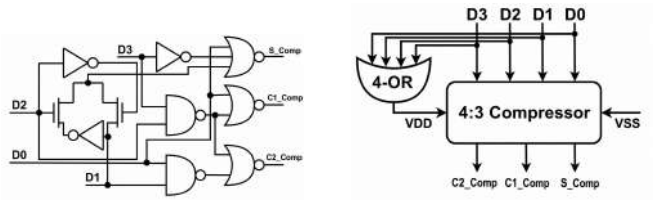


Fig. 2: NOR-enabled SRAM bitcell used in Proposed CIM



(a) (b)

Fig. 3: Sparsity-aware compression: (a) partial-product alignment via rearrangement, and (b) 4-input compressor with sparsity-enabled gating for reduced switching and power.

TABLE II: Circuit-Level Comparison of Compressor & Adder

Block	Metric	DIMCA [22]	Proposed	Improvement
Compressor	Transistor	28–32T	28T	Comparable
	Power (μW)	68.75	35.81	47.9%
	Delay (ns)	0.32	0.22	31%
4-bit Adder	Transistor	72T	38T	47%
	Power (μW)	120	95	21%
	PDP	1×	0.88×	12%

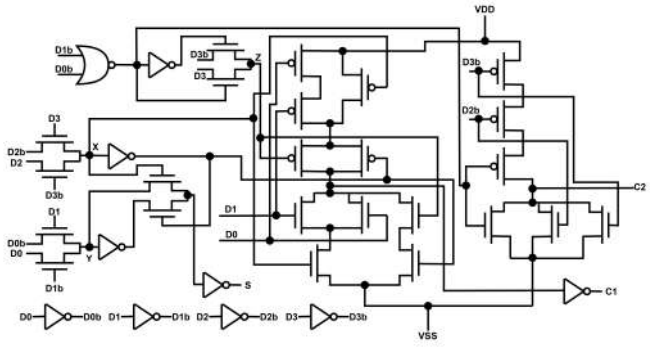


Fig. 4: Proposed 4-bit full adder implementation using a 38T CMOS/PTL logic design.

products are first aligned by the rearrangement network and then processed using 4-input compressors (Fig. 3b). Sparsity-aware gating suppresses redundant switching activity, reducing unnecessary transitions while preserving correct partial-sum accumulation.

The rearrangement network performs column-wise alignment, transforming raw outputs into structured patterns suitable for efficient reduction. In this design, four rearrangement circuits process the four weight-bit positions. The aligned outputs are then compressed using a sparsity-aware 4:3 compressor implemented with NAND/NOR logic and an activity-driven enable signal. When all inputs are zero, the compressor is disabled, reducing dynamic power—particularly beneficial under sparse workloads. To quantify these benefits, Table II compares the proposed design with conventional implementations. The 4:3 compressor uses 28 transistors, comparable to CMOS designs (28–32T) [22], with an additional 6–8 transistors for sparsity detection (EN_comp = D3 OR D2 OR D1 OR D0). Despite this overhead, the design achieves up to 47.9% power savings under sparse conditions. The proposed 4-bit adder further reduces transistor count from 72T to 38T, achieving 21% power reduction and 12% PDP improvement. These results confirm that the proposed circuit-level optimizations significantly improve area and energy efficiency.

D. Compact 4-Bit Adder and Hierarchical Accumulation

The outputs of the compression stage are forwarded to an accumulation backend consisting of a compact 4-bit adder and a hierarchical adder tree. The 4-bit adder is implemented using a 38-transistor CMOS/PTL design, providing low logic depth and compact area for efficient low-bit accumulation (Fig. 4). Functional verification is shown in Fig. 5. A hierarchical 64-bit adder tree (Fig. 6) reduces the compressed outputs through multi-stage accumulation, avoiding long serial carry chains and improving throughput. The rearranged inputs are mapped in reverse order to balance the accumulation path. The 64 inputs are first grouped into sixteen 4:3 compressors. Active compressors generate $(S_{\_Comp}, C1_{\_Comp}, C2_{\_Comp})$, while zero-input compressors remain inactive to reduce switching activity. Representative mappings of the 4:3 compressor from input bits $(D_3$–$D_0)$ to outputs $(C_{2\_Comp}, C_{1\_Comp}, S_{\_Comp})$, e.g., $(1111 \rightarrow 100)$ and $(1110 \rightarrow 011)$, illustrate the reduction of four input bits into sum and carry signals for partial-sum accumulation. The outputs are then organized into three paths (Blocks A, B, and C), each processing 16 inputs. Within each path, the signals are hierarchically combined using 4-bit and 3-bit adders to produce three 5-bit outputs.

In the final stage, these outputs are aligned according to bit significance (Block A: no shift, Block B: 1-bit shift, Block C: 2-bit shift) and accumulated through a structured adder network. The resulting output is an 8-bit vector, with the effective 7-bit sum $(S_6$–$S_0)$ representing the final accumulation result (Fig. 7). This hierarchical design enables high parallelism and reduces critical-path delay compared to ripple-carry accumulation. Post-layout timing results (Table III) show a worst-case delay of 1.24 ns (SS) supporting 800MHz and a typical delay of 1.00 ns (TT), supporting 1 GHz operation without pipelining. The integration of sparsity-aware compression further reduces dynamic power. Overall, the combined use of rearrangement, compression, and hierarchical accumulation forms the key macro-level optimization, enabling efficient and scalable low-bit MAC operations across the 64-bank architecture.

E. Bit-Serial Multiplication and Alignment Stage

To bridge the mismatch between single-bit compressor outputs and multi-bit accumulation, an alignment stage is introduced (Fig. 8). The outputs are registered using D flip-flops (DFFs), and bitwise multiplication between input activations and weight bits is performed using AND gates, enabling bit-serial realization of multi-bit multiplication (e.g., 4-bit × 4-bit) without parallel multipliers. The DFF stage ensures temporal alignment of partial products, which are

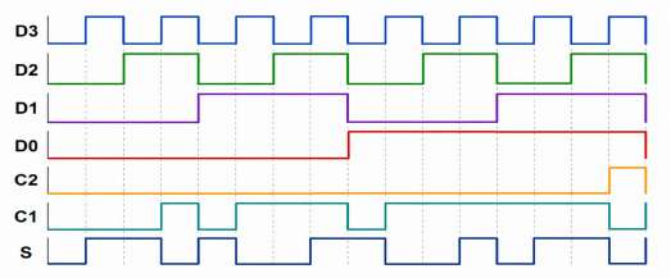


Fig. 5: Functional waveform of the proposed 4-bit full adder.

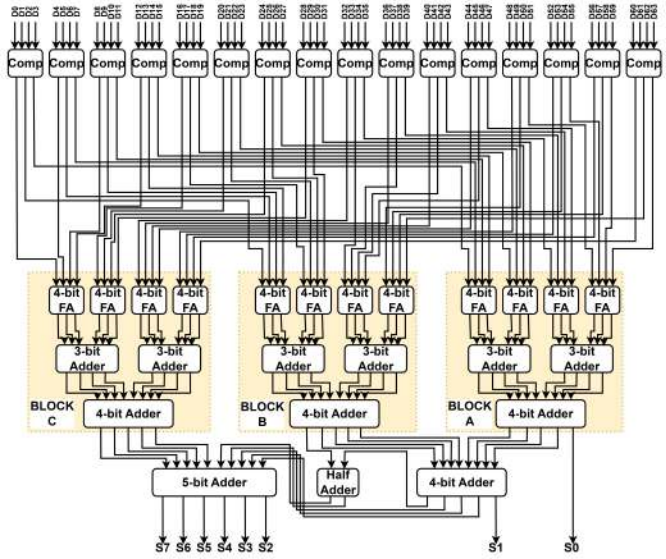


Fig. 6: Proposed hierarchical adder tree for accumulation.

accumulated through shift-and-accumulate operations across cycles, reducing hardware complexity, area, and power. Although the 64×4 array generates multiple weight bit-planes in parallel, the architecture employs time-multiplexed (bit-serial) reduction, where one bit-plane is processed per cycle. This eliminates the need for wide multiplexing or complex selection logic, requiring only lightweight control signals for bit-plane activation. The final result is obtained through sequential shift-and-accumulate operations, ensuring correct bit alignment with minimal overhead.

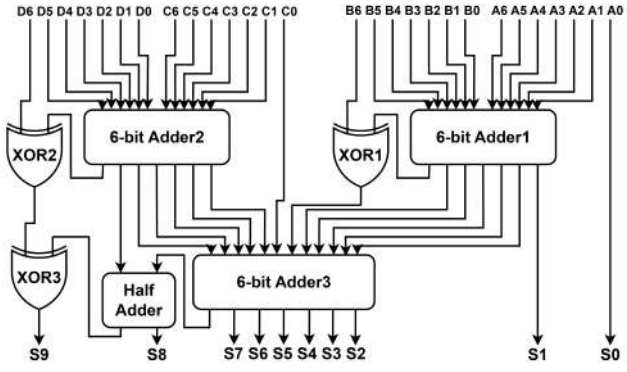


Fig. 7: Shift-and-accumulate (SAC) adder tree using counter-driven iterative accumulation of partial products for in-memory MAC.

TABLE III: Post-Layout Stage-Wise Delay analysis of Hierarchical Adder Tree

Corner	FA Stage (ns)	Adder Stage-1 (3-bit, ns)	Adder Stage-2 (4-bit, ns)	Final Adder (5-bit, ns)	Total (ns)
SS	0.38	0.32	0.42	0.30	1.24
TT	0.27	0.22	0.30	0.21	1.00
FF	0.23	0.18	0.26	0.20	0.87
FS	0.29	0.24	0.33	0.24	1.10
SF	0.31	0.26	0.36	0.27	1.20

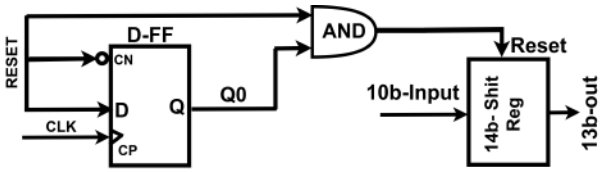


Fig. 8: Bit-serial multiplication and alignment stage using DFF-based temporal alignment and AND-based partial-product generation.

F. Counter-Based Shift-and-Accumulate Engine

The proposed shift-and-accumulate (SAC) module sequentially accumulates partial products using a 13-bit shift register, 4×1 multiplexers, and a 2-bit counter, enabling accurate MAC operations. This structure coordinates preceding outputs and realizes efficient in-memory vector–matrix multiplication.

1) Shift Register and Multiplexer Operation: The 13-bit shift register stores intermediate 10-bit outputs generated from bitwise weight–activation multiplication, followed by compression and adder-tree reduction. Since the weight operand is 4-bit wide, partial products for each weight-bit position are generated sequentially over four cycles. Multiplexers apply the required left shifts before accumulation, as shown in Fig. 9.

2) Shifted Partial Products: Partial products are generated along four bit-significance paths: unshifted (LSB), and left-shifted by 1, 2, and 3 bits for higher weight positions. Here, `0` denotes appended bits due to shifting. For example,

$$\{1'b0,\ q[9:0],\ 2'b0\}$$

represents a 2-bit shift. Counter-driven selection ensures proper alignment prior to accumulation, enabling accurate bit-weighted MAC with minimal overhead.

3) Counter-Based Control for Bit-Serial Accumulation: At each clock cycle, the 2-bit counter increments, and its outputs `COUNT[1:0]` drive the multiplexer select lines (Fig. 9), enabling automatic alignment of partial products according to bit significance without additional control logic. The operation over four cycles is as follows:

- *Cycle 0 (COUNT = 00):* Unshifted partial product (LSB) is accumulated.
- *Cycle 1 (COUNT = 01):* Partial product is left-shifted by 1 bit and accumulated.
- *Cycle 2 (COUNT = 10):* Partial product is left-shifted by 2 bits and accumulated.
- *Cycle 3 (COUNT = 11):* Partial product is left-shifted by 3 bits (MSB) and accumulated, completing the MAC operation (Fig. 10).

This counter-driven multiplexing enables hardware-efficient sequential accumulation of partial products with minimal control overhead while preserving correct bit significance across

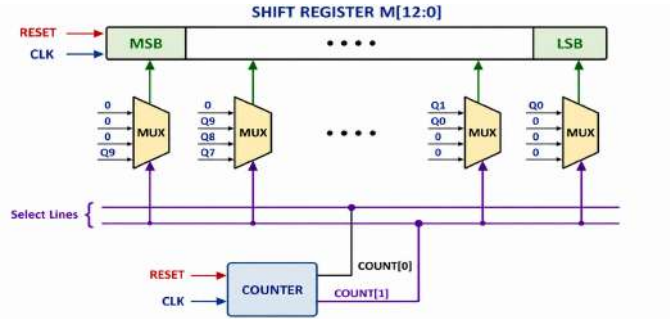


Fig. 9: Bit-shift logic illustrating the interaction among the counter, multiplexers, shift register, and adder.

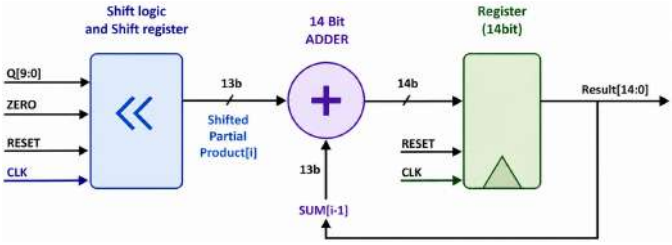


Fig. 10: MAC operation with counter-driven alignment and shared shift-register accumulation.

cycles. For 1-bit activations and 4-bit weights, the MAC result is obtained by accumulating partial products across weight-bit positions. Let the 4-bit weight be expressed as

$$w = \sum_{k=0}^{3} w_k 2^k, \tag{1}$$

where $w_k \in \{0, 1\}$ and $a \in \{0, 1\}$. The multiplication is then as

$$a \cdot w = \sum_{k=0}^{3} (a \cdot w_k) \ll k. \tag{2}$$

Let q_k denote the partial product generated by the adder tree at bit position k. The SAC engine accumulates these as

$$S_k = S_{k-1} + (q_k \ll k), \quad k = 0, 1, 2, 3, \tag{3}$$

with $S_{-1} = 0$, yielding the final result

$$S_3 = \sum_{k=0}^{3} (q_k \ll k). \tag{4}$$

A counter-based control path generates the required shift and synchronizes accumulation, enabling compact multi-bit MAC execution while preserving digital robustness. The same mechanism extends to other low-bit settings by adjusting the number of serial cycles.

IV. Performance Evaluation and Discussion

A. CIM Architecture, Mapping, and Quantization Flow

The proposed SRAM-CIM accelerator consists of 64 banked CIM blocks supporting *1–4*-bit input activations and 4-bit weights. A weight-stationary dataflow is adopted, where weights are stored locally in each bank and input activations are broadcast to enable parallel MAC operations. Each bank independently generates a 14-bit partial sum, which is accumulated using digital logic and quantized to 4-bit precision following [5]. The quantized outputs are stored in local registers and reused for subsequent layer computations.A hardware-aware inference flow is employed to evaluate compatibility with the proposed CIM datapath. Network training is performed in Keras/TensorFlow using 8-bit weights, followed by post-training quantization to 4-bit precision. Input activations are binarized at the hardware interface, and all computations are carried out using integer arithmetic consistent with the CIM architecture.

TABLE IV: Layer-wise mapping of representative CNN and residual CNN workloads onto the proposed 64-bank weight-stationary SRAM-CIM macro.

Network	Layer	Input	Kernel	Output	Ops/pixel	CIM mapping
MNIST on Standard CNNs (LeNet-5)						
LeNet-5	Conv1	32×32×1	5×5×1×6	28×28×6	25	Direct mapping
LeNet-5	Conv2	14×14×6	5×5×6×16	10×10×16	150	Direct mapping
CIFAR-10 on Slandered CNN (VGG-8)						
LeNet-5	Conv1	32×32×3	5×5×3×16	28×28×16	75	Direct mapping
LeNet-5	Conv2	14×14×16	5×5×16×16	10×10×16	400	Direct mapping
VGG-8	Conv1	32×32×3	3×3×3×64	32×32×64	27	Full (64 banks)
VGG-8	Conv3–4	16×16×64	3×3×64×128	16×16×128	576	2 groups (tiling)
VGG-8	Conv5–6	8×8×128	3×3×128×256	8×8×256	1152	4 groups (tiling)
CIFAR-10 on Residual CNN (ResNet-8)						
ResNet-8	ResBlock1	32×32×16	3×3×16×16	32×32×16	144	Direct mapping + skip
	ResBlock2	16×16×32	3×3×32×32	16×16×32	288	Direct mapping + skip
	ResBlock3	8×8×64	3×3×64×64	8×8×64	576	Full (64 banks) + skip

For LeNet-style networks, each output filter is mapped to one CIM bank, such that the number of active banks equals the number of output feature maps. For example, MNIST Conv1 uses 6 banks, while CIFAR-10 employs a modified LeNet configuration with 16 filters, activating 16 banks to accommodate increased input complexity. To demonstrate scalability, the same flow is applied to deeper CNN models, including VGG-8 and ResNet-8. Early layers utilize full 64-bank parallelism, while layers with larger filter counts are supported via filter-group tiling across multiple cycles. Table IV summarizes the layer-wise mapping of representative CNN workloads onto the proposed architecture. Convolution layers are mapped onto the CIM macro, whereas pooling and residual (skip) operations are implemented in the digital domain. Fully connected layers are realized as matrix–vector multiplications with input broadcasting and local weight storage. This unified mapping and quantization flow demonstrates that the proposed banked SRAM-CIM architecture supports both compact and moderately deep CNNs, enabling scalable low-precision inference across MNIST and CIFAR-10 workloads.

B. Effective Throughput and Energy Efficiency

For consistent comparison with prior SRAM-CIM works, both raw peak throughput and effective throughput are reported. In this work, one MAC operation is counted as two operations (one multiplication and one accumulation). Let N_{bank} denote the number of active banks, N_{row} the number of parallel row-wise binary products per bank, B_w the weight precision, C_{ser} the number of serial accumulation cycles, and f_{clk} the clock frequency. The raw throughput captures all bit-level operations generated per cycle prior to serialization and is defined as

$$\text{OPS}_{\text{raw}} = 2 N_{\text{bank}} N_{\text{row}} B_w f_{\text{clk}} \tag{5a}$$

The effective throughput accounts for the serialization overhead due to multi-bit input (1–4-bit) and fixed weight (4-bit)

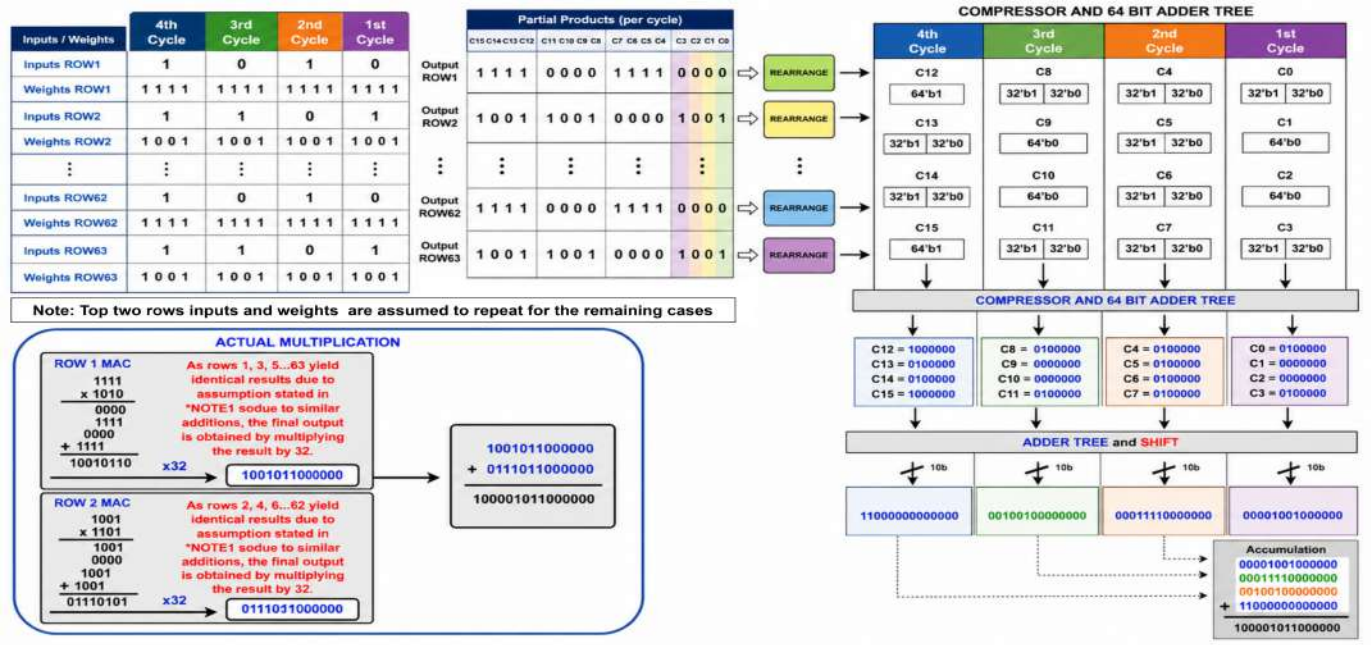


Fig. 11: Dataflow of a 2-D vector dot product using the logical structure of a scaled-down 2-bank macro. The timing waveforms over four cycles illustrate partial-product generation, adder-tree accumulation, and shift-accumulation, producing the final result after five clock cycles.

accumulation and is given by

$$\mathrm{OPS}_{\mathrm{eff}} = \frac{2N_{\mathrm{bank}}N_{\mathrm{row}}B_w f_{\mathrm{clk}}}{C_{\mathrm{ser}}} \tag{5b}$$

For the implemented CNN configuration ($N_{\mathrm{bank}} = 64$, $N_{\mathrm{row}} = 64$, $B_w = 4$, and $C_{\mathrm{ser}} = 4$), substitution into (5a) and (5b) yields

$$\mathrm{OPS}_{\mathrm{raw}} = 32768 f_{\mathrm{clk}} \tag{6a}$$

$$\mathrm{OPS}_{\mathrm{eff}} = 8192 f_{\mathrm{clk}} \tag{6b}$$

The operating frequency of the proposed design is limited by the critical path delay of the hierarchical adder tree, which varies across process corners as summarized in Table IV. At the TT corner, the total delay is 1.00 ns, supporting 1 GHz operation at 1.0 V, while the worst-case SS corner exhibits a delay of 1.24 ns. At post-layout operating points, the achieved raw/effective throughput is summarized as follows:

- 0.9 V: 500 MHz, 4.10 TOPS (TT corner)
- 1.0 V: 1 GHz, 8.19 TOPS (TT corner)
- 1.1 V: 800 MHz, 6.55 TOPS (worst-case across all PVT corners)

The reported 1.1 V operating frequency corresponds to a conservative worst-case value that ensures reliable operation across all PVT conditions. Under typical (TT) conditions, higher operating frequencies are achievable at 1.1 V due to reduced critical path delay, consistent with expected voltage–frequency scaling behavior. This conservative reporting ensures that the presented performance metrics reflect guaranteed operation rather than optimistic peak conditions. Unless otherwise stated, all throughput and energy-efficiency metrics reported in this work are based on $\mathrm{OPS}_{\mathrm{eff}}$, while $\mathrm{OPS}_{\mathrm{raw}}$ is

TABLE V: Summary of Raw and Effective Throughput and Energy Efficiency of the Proposed Macro

V_{DD}	f_{clk} (MHz)	Raw (TOPS)	Eff. (TOPS)	Power (mW)	Eff. (TOPS/W)	Eff. (GOPS/mm^2)	Critical Path Delay(ns)
0.9 V (TT)	500	16.38	4.10	6.99	586	911	2
1.0 V (TT)	1000	32.77	8.19	17.22	476	1820	1.25
1.1 V (all PVT)	800	26.21	6.55	17.66	371	1456	1

TABLE VI: Compressor Power Comparison Under Varying Sparsity

Sparsity (%)	Baseline (μW)	Proposed (μW)	Saving (%)
0	68.75	71.75	-4.4
25	68.75	59.25	13.8
50	68.75	48.31	29.7
70	68.75	38.94	43.4
75	68.75	35.81	47.9

included for completeness and alignment with prior bit-serial CIM designs. Here, energy efficiency is defined as

$$\mathrm{TOPS/W} = \frac{\mathrm{OPS}_{\mathrm{eff}}}{P_{\mathrm{macro}}}, \tag{7}$$

where P_{macro} denotes the total macro power consumption. Using post-layout simulated power values of 6.99 mW, 17.22 mW, and 17.66 mW, the corresponding effective energy efficiencies are 586 TOPS/W, 476 TOPS/W, and 371 TOPS/W at 0.9 V, 1.0 V, and 1.1 V, respectively. These correspond to energy costs of approximately 1.71 fJ/op, 2.10 fJ/op, and 2.69 fJ/op.

C. *Timing, Voltage Scaling, and Sparsity Results*

To enable sparsity-aware operation, a lightweight detection circuit is integrated within each compressor block, defined as $EN_{\mathrm{comp}} = D3 \vee D2 \vee D1 \vee D0$. This signal identifies all-zero input conditions and disables the internal switching activity of the compressor when no valid computation is required.

As a result, unnecessary transitions in the compressor and subsequent reduction stages are suppressed, reducing dynamic power consumption. The detection logic introduces a small overhead of approximately 6–8 transistors per compressor; however, this cost is amortized under sparse workloads due to the significant reduction in switching activity. To evaluate the impact of activation sparsity, we compare the dynamic power of the proposed sparsity-aware 4:3 compressor with a conventional baseline design across different sparsity levels, as summarized in Table VI. At low sparsity (0%), the proposed design incurs a small power overhead due to additional gating logic. However, as sparsity increases, significant power savings are achieved, reaching 29.7% at 50% sparsity and 43.4% at 70% sparsity. This trend highlights the effectiveness of the proposed compressor in reducing switching activity by exploiting input sparsity, making it well-suited for typical CNN workloads where activations are inherently sparse. Table V reports throughput and energy efficiency across supply voltages. The effective throughput accounts for 4-cycle serialisation. A peak of 8.19 TOPS and 476 TOPS/W is achieved at 1.0 V, with 1 GHz operation supported by post-layout timing.

The dataflow of a 2-D vector dot-product operation using a scaled-down 2-bank configuration of the macro shown in Fig. 11. The timing waveforms demonstrate the sequential generation of partial products, intermediate accumulation through the adder tree, and final shift-and-accumulate operations. Over four clock cycles, the partial products corresponding to different weight-bit positions are generated and accumulated, producing the final result after five cycles. This example highlights the temporal behavior of the proposed rearrangement–compression accumulation path and confirms the correctness of the multi-cycle MAC operation implemented within the SRAM-CIM datapath. Fig. 12 shows the post-layout power consumption of the proposed macro across different supply voltages and process corners. The results confirm the expected voltage-scaling trend of digital circuits, where reducing the supply voltage significantly lowers dynamic power consumption. At 0.9 V, the macro achieves the lowest power consumption and the highest energy efficiency, whereas operation at 1.1 V provides higher performance at increased power. These results validate the ability of the proposed architecture to operate across multiple voltage levels while maintaining stable functionality and predictable power scaling.

The proposed 64-bank macro is evaluated over a supply-voltage range of 0.9–1.1 V to characterize the trade-off between throughput and energy efficiency. At 1.1 V, the macro consumes 17.66 mW and achieves 6.55 TOPS, corresponding to 371 TOPS/W. At 1.0 V, the macro reaches the highest throughput of 8.19 TOPS at 17.22 mW with an energy efficiency of 476 TOPS/W. When the supply voltage is reduced to 0.9 V, the power decreases to 6.99 mW while the throughput becomes 4.10 TOPS, yielding the highest energy efficiency of 586 TOPS/W. To illustrate the internal computation flow of the proposed architecture, These results highlight two practical operating points. The 1.0 V condition provides maximum computational throughput, whereas the 0.9 V condition offers the highest energy efficiency. Such flexibility is beneficial for edge-AI systems that must dynamically switch between latency-critical and energy-constrained operating modes. Although the nominal supply of the 65-nm process is higher, the 0.9–1.1 V range is intentionally selected to demonstrate near-nominal low-power operation suitable for practical inference deployment. To further evaluate robustness, the macro is analyzed across process corners and temperature variations. The results show the expected increase in power consumption with higher supply voltage and temperature, while preserving correct functionality of the fully digital accumulation path. Overall, the proposed architecture maintains stable operation across practical PVT variations.

D. Hardware-Compatible PTQ Inference Accuracy

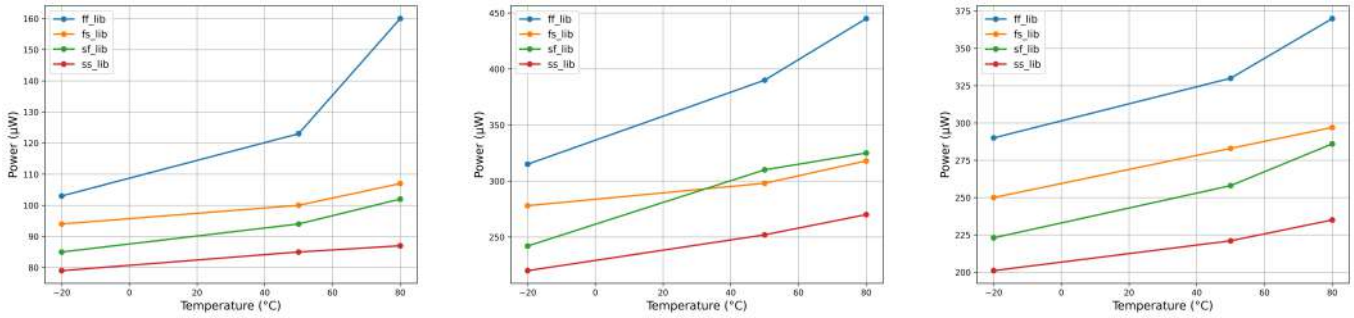
All inference results are obtained using integer arithmetic consistent with the proposed SRAM-CIM datapath. The architecture supports configurable activation precision in the range of 1–4 bits, with 4-bit weights, where the 1-bit activation mode represents the most energy-efficient operating point. Unlike hardware-aware training approaches, the proposed flow does not incorporate quantization effects during training, but strictly enforces hardware constraints during inference. For the LeNet-style network, binarized activations and 4-bit weights are used to evaluate baseline hardware compatibility. The resulting inference accuracy reaches 98.1% on MNIST and 72.3% on CIFAR-10. These results confirm that low-bit mapping preserves high inference quality for simple single-channel inputs while maintaining functionality for more complex multi-channel workloads.

For higher activation precision (2–4 bits), the architecture employs bit-serial processing, where each activation bit is applied sequentially and accumulated through shift-and-add operations in the rearrangement and adder-tree stages. This enables accurate multi-bit MAC computation without increasing datapath complexity, at the cost of additional cycles. To evaluate scalability, we extend the analysis to VGG-8 and ResNet-8 using 4-bit weights and 4-bit activations. The quantized inference results achieve classification accuracies of 88% and 87% on CIFAR-10 for VGG-8 and ResNet-8, respectively. These results are obtained using post-training quantization (PTQ) without retraining, with an accuracy degradation of approximately 3–4% compared to floating-point (FP32) baselines. Overall, the results demonstrate that the proposed low-precision CIM design maintains competitive accuracy while enabling efficient hardware implementation across a range of CNN models and precision configurations.

E. Comparison with Prior SRAM-CIM Macros

A comparison of the proposed HierCIM macro with representative SRAM-based CIM accelerators is summarized in Table VII. The comparison spans technology node, compute style, cell type, supported precision, macro area, benchmark model, accuracy, and throughput. The prior works include recent digital and mixed-signal SRAM-CIM implementations reported in TCAS-I'24 [12], TCAS'25 [15], JSSC'19 [33], JETCAS'22 [34], TNANO'23 [35], JSSC'21 [19], and JSSC'24 [22]. It should be noted that several prior works

[0]The equivalent macro area (*) is quadratically normalized across technology nodes, while the throughput (†) is scaled to CMOS 65 nm using scaling factors reported in [12], [19], [33]–[35].



(a) Power comparison at 0.9 V (b) Power comparison at 1.0 V (c) Power comparison at 1.1 V

Fig. 12: Power comparison of the macro at 0.9V, 1.0V, and 1.1V across process corners. Lowering the supply to 0.9V reduces power and improves energy efficiency, whereas 1.1V provides higher performance at increased power.

TABLE VII: Comparison of the proposed Digital CIM Macro with prior SRAM-based CIM macros.

Parameter	**TCAS-I'24 [12]**	**TCAS'25 [15]**	**JSSC'19 [33]**	**JETCAS'22 [34]**	**TNANO'23 [35]**	**JSSC'21 [19]**	**JSSC'24 [22]**	**This Work**
Technology (nm)	55	65	65	65	65	65	28	65
MAC operation	Digital	Digital	AMS	Analog	Digital	Digital	Digital	Digital
Cell type	8T	6T	10T	AND8T	10T	NA	8T	6T+4T
Supply voltage (V)	1.2	1	0.8–1.0	1.0	1.2	0.6–0.8	0.45–1.1	0.9–1.1
Array size	64 Kb	64 Kb	16 Kb	16 Kb	16 Kb	16 Kb	16 Kb	16 Kb
Operating frequency (MHz)	200	400	5	100	25	138	250	500 / 1000 / 800†
Macro area (mm^2)	2.8	0.3136	–	–	–	0.23	0.49, 2.84	4.5
Macro area ($A_{eq,65}$)*	3.9	0.3136	–	–	–	0.23	2.64, 15.3	4.5
Input precision (bit)	4/8/12/16	1–8	6	4	4	1–16	1–4	1
Weight precision (bit)	4/8/12/16	1–8	1	4	1–4	1–16	1	1–4
Benchmark model	ResNet-10	–	LeNet-5	VGG-8	CNN-type	LeNet-5	VGG-8	LeNet-5
Accuracy (%)	93.7	–	98.3	96.05	98.67	99.2	86.96	98.1 / 72.3
Performance								
Effective TOPS	1.28†	–	0.64	0.41	0.82	0.56	4.8, 2.97†	8.19
Energy Efficiency (TOPS/W)	66.3†	249.1†	51.3	301.08	273	117.3	154	476
Area Efficiency (GOPS/mm^2)	288†	819.2†	513	639.68	–	–	607†	1820

report measured silicon implementations, while the results of this work are based on post-layout simulations with physical-layout validation. Therefore, the comparison positions the proposed architecture within the design space of recent SRAM-CIM macros rather than claiming universal superiority across all technology nodes and evaluation conditions.

For HierCIM, the reported operating frequencies correspond to the post-layout operating points at supply voltages of 0.9 V, 1.0 V, and 1.1 V, respectively. The reported accuracy values correspond to hardware-aware low-bit inference evaluated on the MNIST and CIFAR-10 datasets. From the comparison, HierCIM achieves a peak throughput of 8.19 TOPS, which is the highest among the listed 65-nm SRAM-CIM macros and remains competitive even against designs implemented in more advanced technology nodes [12], [19], [22], [36], as shown in Table VII. Using the normalized values in Table VII, the proposed macro provides approximately 6.40×, 5.06×, 12.80×, 19.98×, 9.99×, 14.63×, and 2.76× higher throughput than TCAS-I'24 [12], TCAS'25 [15], JSSC'19 [33], JETCAS'22 [34], TNANO'23 [35], JSSC'21 [19], and JSSC'24 [22], respectively. These improvements mainly arise from the proposed rearrangement–compression accumulation path, which reduces redundant switching activity and shortens the multi-stage accumulation critical path while enabling efficient 64-bank parallelism. Table VII distinguishes reported macro area from 65-nm-equivalent area and separates raw peak throughput from effective throughput. For fair cross-technology comparison (e.g., 55 nm, 65 nm, 28 nm), we report both the original area and a normalized area, obtained via quadratic scaling to 65 nm:

$$A_{\mathrm{eq},65} = A_{\mathrm{reported}} \left(65/L_{\mathrm{tech}}\right)^2 \tag{8}$$

where A_{reported} is the reported area and L_{tech} is the technology node (nm). This scaling captures the first-order dependence of silicon area on feature size. Table VII lists both reported and normalized areas to ensure consistent area-efficiency comparison. For the proposed design, the raw peak throughput is 32.77 TOPS at 1.0 V. Considering four serial weight-bit accumulation cycles, the effective throughput is 8.19 TOPS, representing sustained performance. Accordingly, all comparisons use effective throughput and area efficiency unless prior work reports only alternative metrics.

Another important advantage of HierCIM is its fully digital compute architecture. Several prior CIM designs rely on analog or mixed-signal computation using current summation or ADC-based sensing [33], [34], which may introduce non-linearity, sensing noise, and calibration overhead. In contrast, the proposed architecture performs MAC operations using purely digital logic. This improves robustness and compatibility with standard CMOS design flows while still delivering strong throughput and energy efficiency. In addition, HierCIM supports configurable low-bit computation with 1–4-bit weights and is evaluated with binarized activations for

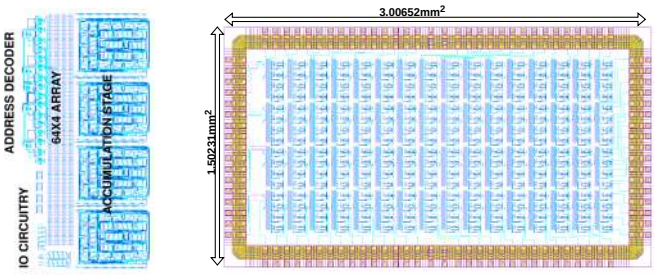
edge-oriented CNN inference. Compared with prior digital CIM macros that focus on single-bit weights [33] or wider but fixed precision configurations [12], [19], the proposed design provides a practical operating range for quantized neural networks. This makes the architecture particularly attractive for compact edge-AI systems that require a favorable balance among throughput, accuracy, and implementation efficiency.

Although the proposed 6T+4T compute-enabled cell introduces additional hardware overhead compared with conventional SRAM cells, the resulting macro area of 4.5 mm^2 should be interpreted together with the achieved compute capability. In this design, the additional hardware enables direct in-memory logic evaluation and a simplified arithmetic dataflow, which together support significantly higher throughput and a scalable banked organization for efficient CNN mapping. Finally, the proposed macro is validated using LeNet-5 inference and achieves 98.1% accuracy on MNIST and 72.3% on CIFAR-10 under the reported low-bit configuration. While some prior works report higher accuracy under different datasets, quantization schemes, or benchmark conditions [19], [34], the objective of this work is to demonstrate a circuit–architecture co-design optimized for efficient low-bit digital inference. Overall, the results indicate that the proposed rearrangement-assisted compression and hierarchical accumulation architecture enables substantial throughput improvement while preserving the robustness and scalability advantages of digital SRAM-CIM implementations.

F. Physical Layout and Area Breakdown

The physical implementation of the proposed architecture is shown in Fig. 13. Fig. 13a presents the layout of the compute-enabled 6T+4T SRAM bit-cell with its peripheral circuitry, while Fig. 13b illustrates the layout of a representative CIM bank including local interconnects and peripheral logic. The layout demonstrates that the compute logic can be integrated into the SRAM array while preserving a regular memory-centric structure, enabling scalable replication of the bank architecture for larger arrays. To provide a physically grounded evaluation, the proposed architecture is implemented with explicit layouts of the compute-enabled cell and a representative CIM bank. The layout preserves a regular memory-centric organization in which the SRAM array forms the dominant structure, while the rearrangement, compression, and arithmetic logic are placed at the periphery to simplify routing and clock distribution. The compute-enabled 6T+4T cell occupies 1.2 μm^2 in 65-nm technology.

The area distribution of the proposed macro is summarized in Figure 14. The SRAM array occupies the largest portion of the macro area, accounting for approximately 40% of the total footprint. The rearrangement and compression logic contributes about 10%, while the hierarchical adder tree occupies roughly 35%. The remaining 15% is allocated to the SAC and control circuitry. This breakdown highlights the architectural trade-off between embedding computation inside the memory array and the additional digital logic required to support efficient in-memory accumulation. A representative CIM bank occupies 0.0703 mm^2, and the complete 64-bank macro occupies 4.5 mm^2. The macro area is distributed across four main components: the SRAM array, rearrangement and compression



(a) (b)

Fig. 13: Physical layout of the proposed SRAM-based DCIM macro: (a) NOR-enabled 6T+4T compute cell with accumulation stage & peripheral circuitry and (b) CIM bank.

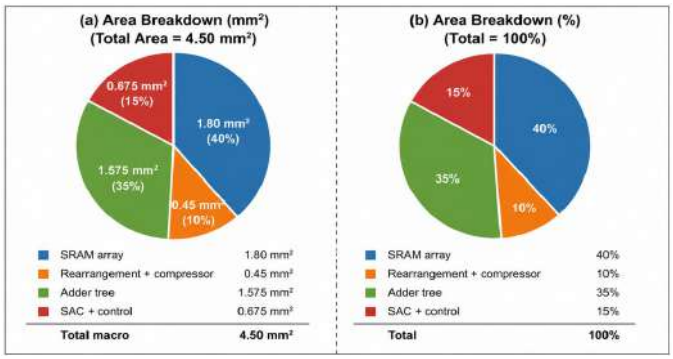


Fig. 14: Area breakdown of the proposed SRAM-CIM macro

logic, hierarchical adder tree, and SAC/control circuitry. These blocks account for approximately 40%, 10%, 35%, and 15% of the total macro area, respectively. This breakdown highlights the design trade-off introduced by enabling computation inside the memory array. Although the proposed design increases the bit-cell area relative to a conventional SRAM cell, it significantly reduces off-array data movement and enables an efficient in-memory reduction path for low-bit inference. The regular banked structure also allows the architecture to scale by replicating bank modules for larger arrays.

V. Conclusion

This work presents a 16-Kb, 64-bank, all-digital SRAM-based CIM macro for low-bit CNN inference. The key contribution is a macro-level co-design of a compute-enabled SRAM cell, rearrangement network, sparsity-aware compression, compact arithmetic, and hierarchical accumulation within a unified banked architecture. The design supports configurable 1–4-bit activations and 1–4-bit weights, enabling an efficient balance between robustness and energy efficiency. Implemented in 65-nm CMOS, the macro achieves 32.77 TOPS raw throughput and 8.19 TOPS effective throughput at 1.0 V, with a peak effective energy efficiency of 586 TOPS/W at 0.9 V. The effective throughput reflects practical factors such as multi-cycle accumulation and utilization-aware mapping. Hardware-aware evaluation demonstrates 98.1% accuracy on MNIST and 72.3% on CIFAR-10 for LeNet with 1-bit activations and 4-bit weights. Extended evaluation on VGG-8 and ResNet-8 using 4-bit precision achieves 88% and 87% accuracy on CIFAR-10, with 3–4% degradation from

FP32 baselines. All results are obtained using a hardware-compatible post-training quantization (PTQ) flow without retraining. Overall, the proposed SRAM-CIM architecture offers a practical trade-off among throughput, energy efficiency, and implementability, making it well suited for edge-oriented DNN inference and scalable to other CNN workloads.

ACKNOWLEDGEMENT

The authors acknowledge the MeitY, Government of India, under the Chips to Startup (C2S) Project, for financial and EDA tool support. The authors also thank the System-on-Chip Lab, part of the Cyber-Physical Systems Center at Khalifa University, UAE, for research infrastructure and support.

REFERENCES

[1] W. G. Hatcher and W. Yu, "A survey of deep learning: Platforms, applications and emerging research trends," IEEE Access, vol. 6, pp. 24 411–24 432, 2018.

[2] H. Amrouch, N. Du, A. Gebregiorgis, S. Hamdioui, and I. Polian, "Towards reliable in-memory computing: From emerging devices to post-von-neumann architectures," in 2021 IFIP/IEEE 29th International Conference on Very Large Scale Integration (VLSI-SoC). IEEE, 2021, pp. 1–6.

[3] S. Lee et al., "A 1ynm 1.25V, 8Gb, 16Gb/s/pin GDDR6-based accelerator-in-memory supporting 1TFLOPS MAC operation and various activation functions for deep-learning applications," in 2022 IEEE International Solid-State Circuits Conference (ISSCC), vol. 65. IEEE, 2022, pp. 1–3.

[4] S.-H. Sie et al., "MARS: Multimacro architecture SRAM CIM-based accelerator with co-designed compressed neural networks," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 41, no. 5, pp. 1550–1562, 2022.

[5] B. Yan et al., "A 1.041-Mb/mm^2 27.38-TOPS/W signed-INT8 dynamic-logic-based adc-less sram compute-in-memory macro in 28nm with reconfigurable bitwise operation for ai and embedded applications," in 2022 IEEE International Solid-State Circuits Conference (ISSCC), vol. 65. IEEE, 2022, pp. 188–190.

[6] S. S. Kumar and J. Nayak, "Effective 8t reconfigurable SRAM for data integrity and versatile in-memory computing-based AI acceleration," Electronics, vol. 14, no. 13, p. 2719, 2025.

[7] A. Agrawal et al., "XCEL-RAM: Accelerating binary neural networks in high-throughput SRAM compute arrays," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 66, no. 8, pp. 3064–3076, 2019.

[8] Z. Chen et al., "CAP-RAM: A charge-domain in-memory computing 6T-SRAM for accurate and precision-programmable CNN inference," IEEE Journal of Solid-State Circuits, vol. 56, no. 6, pp. 1924–1935, 2021.

[9] L. Sonnino, S. Shresthamali, Y. He, and M. Kondo, "DAISM: Digital approximate in-SRAM multiplier-based accelerator for DNN training and inference," in 2024 Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE, 2024, pp. 1–6.

[10] C. Eckert et al., "Neural cache: Bit-serial in-cache acceleration of deep neural networks," in 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA). IEEE, 2018, pp. 383–396.

[11] M. Cheshfar et al., "Comparative survey of embedded system implementations of convolutional neural networks in autonomous cars applications," IEEE Access, 2024.

[12] W. Yi et al., "RDCIM: RISC-V supported full-digital computing-in-memory processor with high energy efficiency and low area overhead," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 71, no. 4, pp. 1719–1732, 2024.

[13] N. D. Lane et al., "An early resource characterization of deep learning on wearables, smartphones and internet-of-things devices," in Proceedings of the 2015 International Workshop on Internet of Things towards Applications, 2015, pp. 7–12.

[14] M. Ali et al., "Compute-in-memory technologies and architectures for deep learning workloads," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 30, no. 11, pp. 1615–1630, 2022.

[15] V. Sharma, X. Zhang, N. S. Dhakad, and T. T.-H. Kim, "FlexDCIM: A 400 MHz 249.1 TOPS/W 64 kb flexible digital compute-in-memory sram macro for cnn acceleration," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 72, no. 10, pp. 5696–5707, 2025.

[16] J. Oh, C.-T. Lin, and M. Seok, "D6cim: 60.4-TOPS/W all-digital 6T-SRAM-based compute-in-memory macro supporting 1-to-8 b fixed-point arithmetic in a 28-nm cmos," IEEE Transactions on Circuits and Systems I: Regular Papers, pp. 1–12, 2026.

[17] W. Lu et al., "A floating-point sram computing-in-memory macro using digital-domain structure for cnns," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, pp. 1–10, 2026.

[18] K. Murugan, R. Nithya, K. Prasanth, S. Fowjiya, R. U. Mageswari, and E. A. M. Ali, "Analysis of full adder cells in numerous logic styles," in 2022 International Conference on Electronics and Renewable Systems (ICEARS). IEEE, 2022, pp. 90–96.

[19] H. Kim, T. Yoo, T. T.-H. Kim, and B. Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," IEEE Journal of Solid-State Circuits, vol. 56, no. 7, pp. 2221–2233, 2021.

[20] Q. Zhang et al., "Lightweight and accurate DNN-based anomaly detection at edge," IEEE Transactions on Parallel and Distributed Systems, vol. 33, no. 11, pp. 2927–2942, 2021.

[21] A. Sadeghi, N. Shiri, and M. Rafiee, "High-efficient, ultra-low-power and high-speed 4:2 compressor with a new full adder cell for bioelectronics applications," Circuits, Systems, and Signal Processing, vol. 39, no. 12, pp. 6247–6275, 2020.

[22] C.-T. Lin et al., "Dimca: An area-efficient digital in-memory computing macro featuring approximate arithmetic hardware in 28 nm," IEEE Journal of Solid-State Circuits, vol. 59, no. 3, pp. 960–971, 2023.

[23] B. Zhang et al., "MACC-SRAM: A multistep accumulation capacitor-coupling in-memory computing sram macro for deep convolutional neural networks," IEEE Journal of Solid-State Circuits, vol. 59, no. 6, pp. 1938–1949, 2023.

[24] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," IEEE Journal on Emerging and Selected Topics in Circuits and Systems, vol. 9, no. 2, pp. 292–308, 2019.

[25] Z. Sun and R. Huang, "Time complexity of in-memory matrix-vector multiplication," IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 68, no. 8, pp. 2785–2789, 2021.

[26] K. Yoshioka, S. Ando, S. Miyagi, Y.-C. Chen, and W. Zhang, "A review of SRAM-based compute-in-memory circuits," Japanese Journal of Applied Physics, 2024.

[27] N. Ashar, G. Raut, V. Trivedi, S. K. Vishvakarma, and A. Kumar, "QuantMAC: Enhancing hardware performance in DNNs with quantize-enabled multiply-accumulate unit," IEEE Access, vol. 12, pp. 43 600–43 614, 2024.

[28] G. Raut et al., "Designing a performance-centric MAC unit with pipelined architecture for DNN accelerators," Circuits, Systems, and Signal Processing, vol. 42, no. 10, pp. 6089–6115, 2023.

[29] G. Raut, S. Karkun, and S. K. Vishvakarma, "An empirical approach to enhance performance for scalable CORDIC-based deep neural networks," ACM Transactions on Reconfigurable Technology and Systems, vol. 16, no. 3, pp. 1–32, 2023.

[30] S. Vishwakarma et al., "A precision-aware neuron engine for DNN accelerators," SN Computer Science, vol. 5, no. 5, p. 494, 2024.

[31] N. S. Dhakad et al., "In-memory computing with 6T SRAM for multi-operator logic design," Circuits, Systems, and Signal Processing, vol. 43, no. 1, pp. 646–660, 2024.

[32] Y.-C. Chiu et al., "A 4-kb 1-to-8-bit configurable 6T SRAM-based computation-in-memory unit-macro for cnn-based ai edge processors," IEEE Journal of Solid-State Circuits, vol. 55, no. 10, pp. 2790–2801, 2020.

[33] A. Biswas and A. P. Chandrakasan, "CONV-SRAM: An energy-efficient SRAM with in-memory dot-product computation for low-power convolutional neural networks," IEEE Journal of Solid-State Circuits, vol. 54, no. 1, pp. 217–230, 2019.

[34] V. Sharma, J.-E. Kim, H. Kim, L. Lu, and T. T.-H. Kim, "A reconfigurable 16kb AND8T SRAM macro with improved linearity for multi-bit compute-in-memory of artificial intelligence edge devices," IEEE Journal on Emerging and Selected Topics in Circuits and Systems, vol. 12, no. 2, pp. 522–535, 2022.

[35] P. K. Saragada, S. Manna, A. Singh, and B. P. Das, "A configurable 10t SRAM-based IMC accelerator with scaled-voltage-based pulse count modulation for MAC and high-throughput XAC," IEEE Transactions on Nanotechnology, vol. 22, pp. 222–227, 2023.

[36] J. Oh et al., "D6cim: 60.4-TOPS/W, 1.46-TOPS/mm^2, 1005-kb/mm^2 digital 6T-sram-based compute-in-memory macro supporting 1-to-8b fixed-point arithmetic in 28-nm cmos," in IEEE European Solid-State Circuits Conference (ESSCIRC), vol. 49. IEEE, 2023, pp. 413–416.